AI-Powered Real Estate Property Triage System

Final Project Book — AI Engineering Course

Author: Yehuda Rokach **Project type:** Individual final project **Date:** June 2026 **Status:** Phases 0–3 complete — WebUI + 4 microservices built, trained, and tested; n8n orchestration + EC2 deployment remaining

Table of Contents

1. Abstract

This project designs and builds a production-style, multi-modal AI system that automates the intake and initial evaluation of real-estate property listings for a fictional agency. A listing agent submits a free-text description plus property photos; the system validates the submission, extracts structured fields, classifies each image by room type and condition, retrieves comparable past listings, and produces a ready-to-publish listing brief — then routes it to the correct team.

The system is assembled from managed and reusable building blocks rather than hand-rolled from scratch: **Amazon Bedrock Knowledge Bases** for retrieval, **Amazon Bedrock Guardrails** for safety, **Google Gemini** for all LLM generation, a **PyTorch** transfer-learning model for image analysis, **n8n** for orchestration, and a **Flask + Ollama** web interface.

As of this revision, the WebUI, all four microservices (RAG, Image Analyser, Guardrails, LangGraph Agent) — covered by a 43-test offline suite — and the **n8n 8-node orchestration flow** are built and validated; the image classifier reaches **84.6%** room-type accuracy on fresh images, and the full pipeline passes both guardrails with correct residential/commercial routing — verified end-to-end from the WebUI through n8n and back. EC2 deployment remains.

2. Introduction & Real-World Scenario

A real-estate agency receives dozens of new property submissions every day, each a written description plus photographs. Staff must check the submission is genuine (not spam/off-topic), identify property type/condition/features, score the uploaded images, find similar past listings, route the listing to residential vs. commercial teams, and produce a clean published brief.

This is a realistic, multi-modal workflow: it combines text understanding, image analysis, retrieval-augmented generation, safety filtering, and agent-based reasoning in one coherent product. This project automates the entire pipeline end-to-end.

Learning objectives demonstrated: deploying a multi-service AI system on cloud infrastructure; building n8n automation flows with AI nodes; a retrieval-augmented pipeline; a PyTorch image classifier; prompt engineering across six distinct surfaces; input/output safety guardrails; a multi-step reasoning agent; and a local-LLM conversational UI.

3. System Architecture

The system has four layers, each communicating with the next over HTTP.

Layer 1 — Web UI (HTML/CSS/JS + Flask, local). Two working surfaces plus a monitoring extension: (a) a conversational assistant backed by a **local Ollama** server running Llama 3.1, grounded as a real-estate assistant; (b) a listing submission form that POSTs to the n8n webhook and renders the returned brief; (c) a monitoring dashboard with live processing stats (Chart.js). Built as a custom Flask app (instructor permits) for full design control; all the Python logic is

shared.

Layer 2 — n8n orchestration. An 8-node flow: webhook trigger → guardrails input check → IF (pass/fail) → Information Extractor (Gemini) → AI Agent (Gemini; dispatches tool calls to the services) → LLM Chain (final brief) → guardrails output check → router (residential vs. commercial).

Layer 3 — FastAPI microservices. Four independent, containerised services: RAG, Image Analyser, Guardrails, and a LangGraph Agent. Each exposes a single well-defined endpoint.

Layer 4 — Managed services & external LLM. Amazon Bedrock Knowledge Bases (Titan V2 embeddings on **S3 Vectors**) and Amazon Bedrock Guardrails; Google Gemini for all text generation; Ollama (local) for the assistant chat.

A full architecture diagram and the intentional deviations from the guideline are in `docs/architecture.md`.

4. Technology Choices

This project intentionally **assembles managed services** instead of hand-rolling each component, which is both the instructor's intent (a few hours of assembly) and good engineering for a solo build.

Two AWS Bedrock services (requirement: ≥2). - **Bedrock Knowledge Bases** powers the RAG service — managed vector store, embeddings, and retrieval, reusing a working pattern the author already built. - **Bedrock Guardrails** powers the Guardrails service — managed content filters, PII handling, and contextual grounding.

Gemini for all LLM generation. Rather than adding a third Bedrock dependency, all generation (RAG insight, guardrail classification/auditing, the agent's reasoning, and the n8n nodes) uses Google Gemini, which the author already uses fluently. The Gemini API key is stored in **AWS Secrets Manager** and pulled by each service using its existing AWS credentials — no secret in the repo.

Intentional deviations from the written guideline. The guideline lists NeMo Guardrails, Llama.cpp, and a local ChromaDB. We substitute **Bedrock Guardrails** (for NeMo) and **Bedrock Knowledge Bases** (for Llama.cpp + ChromaDB). This satisfies the ≥2-Bedrock-services requirement, reduces operational burden for a solo developer, and — as a side effect — already satisfies the "managed vector store" optional extension.

PyTorch transfer learning (EfficientNet-B0) for the Image Analyser, because the rubric explicitly rewards a trained model.

Component	Technology
Web UI	HTML/CSS/JS + Flask

Conversational LLM	Ollama + Llama 3.1 (local)
Orchestration	n8n
LLM generation	Google Gemini
RAG	Bedrock Knowledge Base (Titan V2 on S3 Vectors)
Image Analyser	PyTorch + EfficientNet-B0 (7 classes incl. `not_a_room`)
Guardrails	Bedrock Guardrails (`ApplyGuardrail`) + Gemini
Agent	LangGraph + Gemini
Secrets	AWS Secrets Manager

5. Component Design

5.1 Web UI (built first)

A 3-tab app served by **Flask** with a vanilla HTML/CSS/JS frontend. **Assistant** tab streams chat from the local Ollama `/api/chat` endpoint with a real-estate system prompt that refuses off-topic questions, gives no legal advice, and invents no prices (prompt surface #5). **Submit Listing** tab posts the description, images, and agent name to the n8n webhook and renders the returned brief, image condition scores, and similar listings. **Dashboard** tab shows live stats (Chart.js) from a local event log. During early development the submit tab is tested against a mock brief so the whole UI is demoable before n8n exists.

5.2 RAG service (`POST /query`)

Embeds the description and retrieves the top-3 most similar past listings from a Bedrock Knowledge Base pre-populated with ≥20 synthetic listings, then generates a short insight with Gemini that cites the listing it drew from and never fabricates facts. Output: `{ similar_listings, insight }`.

5.3 Image Analyser (`POST /analyse`)

A transfer-learning EfficientNet-B0 (ImageNet weights, frozen backbone, retrained head) classifying **7 classes**: kitchen, bathroom, living room, bedroom, exterior, other, and a **`not_a_room`** reject class (added beyond the spec) so non-property photos are flagged rather than forced into a

room. Below a 0.55 confidence threshold it returns `uncertain`. Output: `{ room_type, condition_score, confidence }`. Trained on public Kaggle datasets pulled via `kagglehub` (500 images/class; rooms from *robinreni/house-rooms*, exterior from *mikhailma* street data, negatives from *prasunroy/natural-images*). The **condition score is a documented placeholder** for now — room datasets carry no condition ground truth; a second head (labels bootstrapped with Gemini Vision) is future work. Full evaluation in docs/model_card.md.

5.4 Guardrails service (`POST /check/input`, `POST /check/output`)

Input check: Bedrock `ApplyGuardrail` for safety (content filters, denied topics, profanity) plus a Gemini classifier that accepts only genuine property listings in the expected language (rejecting others, including unexpected languages — the multilingual extension). Output check: Bedrock safety plus a Gemini factuality-vs-source check that catches invented prices, fabricated certifications, and false legal claims. Output: `{ pass, reason, safe_text }`. (PII masking was removed — the system has no email/phone fields — and `_apply_guardrail` fails *closed* on any intervention.)

5.5 LangGraph Agent (`POST /agent/run`)

A 3-node state graph — planner → tool executor → synthesiser — using Gemini, where the executor calls the RAG and Image Analyser services. Answers multi-step questions (e.g., "what renovation work would bring this property to condition score 5?"). Output: `{ answer, tools_used, reasoning_steps }`.

5.6 n8n flow

Eight nodes wiring the webhook through the guardrails, the Gemini Information Extractor and AI Agent, the LLM Chain that produces the brief, the output guardrail, and a router that sends residential vs. commercial listings to different teams.

6. Prompt Engineering

Six prompt surfaces are tuned and logged with ≥10 test cases and a measured pass rate per surface: the n8n Information Extractor, the n8n AI Agent prompt, the RAG insight/citation prompt, the Guardrails rail prompts, the Ollama system prompt, and the LangGraph Agent tool descriptions. The full iteration history is in docs/prompt_log.md.

Done so far: Surface #3 (RAG insight) — V1 = 10/10, **0 fabricated listing ids**; Surface #4 (Guardrails) — V1 = 11/11, 0% false positives on valid listings; Surface #5 (Ollama) — iterated V1→V6 to 10/10 (key fixes: forbidding invented URLs/prices, anti-prompt-injection guards, in-code language matching, listings-awareness); Surface #6 (Agent tool descriptions) — V1 baseline, routing verified live. Surfaces #1–#2 are captured when the n8n flow is built (Phase 4).

7. Results & Evaluation

Image Analyser. Room-type accuracy on fresh, unseen images: **84.6%** (argmax, 40/class) — clears the >75% bar. Per-class: exterior 100%, bathroom 88%, bedroom 85%, kitchen 85%, living_room 80%, other(dining) 70%. The `not_a_room` reject class correctly flags dogs, cats, documents, and diagrams (confidence 0.73–0.95) where the 6-class model was overconfidently wrong. 7-class validation accuracy 84.4%. Full confusion matrix + OOD analysis in `docs/model_card.md`.

Guardrails. Surface #4 V1: 11/11 correct decisions, 0% false positives on the valid-listing set; spam/off-topic/offensive all blocked; non-Hebrew/English rejected with a localized message.

RAG. Surface #3 V1: 10/10, 0 fabricated listing ids (regex-verified), citations in every insight.

Testing. A 43-test offline `pytest` suite (AWS/Gemini/Ollama mocked) covers the shared helpers, all four services, and the WebUI — re-runnable with no credentials or network.

Still to do: RAG precision@3 benchmark (managed-vector-store extension), and an end-to-end run with screenshots once n8n is wired.

8. Deployment Notes

The four FastAPI services are deployed on **AWS EC2** via Docker Compose and were verified end-to-end.

- **Instance:** t3.large, Amazon Linux 2023, `us-east-1`, 30 GB gp3 root.
- **Bootstrap** (`deploy/ec2-userdata.sh`): installs Docker + the Compose plugin, git-clones the public repo, pulls the trained `model.pth` from S3, and runs `docker compose up --build -d` (`docker-compose.yml` at the project root).
- **Credentials — no keys on the box:** the instance carries an **IAM role**; the services read the Gemini key from **Secrets Manager** and call **Bedrock** (KB `Retrieve` + `ApplyGuardrail`) through it. Resource IDs and the Gemini secret *name* are plain Compose env.
- **Management via SSM:** no SSH key and **no public inbound** (the security group has no ingress rules) — the instance is driven through AWS Systems Manager.
- **Verified on EC2:** all four `/health` returned ok (image `model_loaded: true` from the S3 model), and a live RAG `/query` returned real KB comparables (L010/L002/L001) plus a grounded Gemini insight — proving the role-based AWS access works end-to-end.
- **Cost control:** the instance is **stopped when idle** (no compute charge; containers auto-resume on start via `restart: unless-stopped`).
- **Docker on Apple Silicon:** build `--platform linux/amd64` for parity with the x86 EC2 host.
- **n8n:** runs in Docker locally for development; for a fully-hosted demo it can run on the same instance with its Gemini credential configured once.

9. Conclusions & Future Work

To be written at the end. Candidate future work: the human-in-the-loop feedback/active-learning loop (deferred), a managed Pinecone/Weaviate comparison, and richer monitoring.

10. References

- Project guideline: *AI Engineering — Final Project: AI-Powered Real Estate Property Triage System.*
- Amazon Bedrock Knowledge Bases, Amazon Bedrock Guardrails (AWS documentation).
- Google Gemini API documentation.
- PyTorch transfer-learning tutorial; torchvision model zoo.
- n8n documentation; Ollama documentation; Streamlit documentation.